

DEVELOPER SPECIFICATION · VERSION 1.0.0

Brokerage REST API

Accounts, positions, order status, transaction history, and stock and options trading — over an OAuth 2.1 protected, hypermedia-driven REST surface.

IN THIS DOCUMENT

Authentication and authorization · OAuth 2.1 + PKCE

Accounts APIs · list, balances, positions, order status, transactions

Trading APIs · stock and options

Worked examples in Java, Node.js, and Python

OpenAPI 3.2.0 specification

Audience Internal development team

Status Draft for review

Companion portal `brokerage-apis/dev-portal/`

Machine-readable spec `brokerage-apis/openapi/brokerage-api.yaml`

Contents

1 Overview

- 1.1 What this API is for, and what it deliberately excludes
 - 1.2 Environments and base URLs
 - 1.3 The design contract at a glance
 - 1.4 Reference implementations we borrowed from
-

2 Authentication and Authorization

- 2.1 Why OAuth 2.1 with PKCE
 - 2.2 Client registration and client types
 - 2.3 The authorization code flow, end to end
 - 2.4 PKCE mechanics: verifier, challenge, and what it defends
 - 2.5 Exchanging the code for a bearer token
 - 2.6 Access tokens: structure, lifetime, and validation
 - 2.7 Refresh tokens, rotation, and reuse detection
 - 2.8 Scopes and least privilege
 - 2.9 Sender-constrained tokens with DPoP
 - 2.10 Revocation, logout, and token hygiene
 - 2.11 Device authorization for input-constrained clients
-

3 API Conventions

- 3.1 Hypermedia and HATEOAS
 - 3.2 Errors: RFC 9457 problem details
 - 3.3 Decimals, money, and why nothing is a float
 - 3.4 Pagination
 - 3.5 Idempotency
 - 3.6 Optimistic concurrency with ETags
 - 3.7 Rate limits and backoff
 - 3.8 Versioning and compatibility
-

4 Accounts APIs

- 4.1 Account list
 - 4.2 Account balances
 - 4.3 Account positions
 - 4.4 Account order status
 - 4.5 Account transaction history
-

5 Trading APIs

- 5.1 The order lifecycle
 - 5.2 Placing an order safely
 - 5.3 Stock trading
 - 5.4 Options trading
 - 5.5 Previewing an order
 - 5.6 Amending and cancelling
 - 5.7 Streaming order events and webhooks
-

6 Code Examples

- 6.1 How the examples are organized
 - 6.2 Authorization: PKCE in three languages
 - 6.3 Accounts: reading balances and positions
 - 6.4 Trading: placing a stock order
 - 6.5 Trading: placing a multi-leg option order
 - 6.6 Amend and cancel with optimistic concurrency
-

7 OpenAPI 3.2.0 Specification

- 7.1 Where the specification lives
 - 7.2 What 3.2.0 gives us that 3.1 did not
 - 7.3 Document structure
 - 7.4 Validating and generating from the specification
-

A Appendix: Error catalogue

B Appendix: Enumerations

C Appendix: Glossary

1 Overview

This document specifies a REST API for the brokerage domain: reading accounts and positions, tracking orders and settled activity, and trading stocks and listed equity options. It is written for the engineers who will build against it, and it states not only what each endpoint does but why it was shaped that way.

1.1 What this API is for, and what it deliberately excludes

The surface covers the operations a trading application actually needs on a customer's behalf: enumerate their accounts, read balances and holdings, place and manage stock and option orders, and reconcile what happened afterwards. Everything is scoped to a single authenticated customer's grant.

Three areas are intentionally out of scope, and no endpoint here touches them:

EXCLUDED	WHY, AND WHERE IT BELONGS INSTEAD
Institutional application process	Account opening, entity onboarding, and suitability review are a compliance workflow, not a trading API. They are governed by different regulation, run on human timescales, and carry PII this surface never handles.
Futures trading	Futures carry a different margin model (SPAN rather than Reg T), different contract identifiers, and different exchange plumbing. Folding them into the same order resource would compromise both.
Crypto trading	Different custody model, 24/7 sessions, no settlement cycle, and a separate regulatory regime. It shares almost nothing with an equity order except the word "buy".

A small **instruments** surface is included even though it was not requested, for a practical reason: an option order references a contract by `instrumentId`, and without a way to resolve `AAPL Dec-18-2026 250 Call` into that identifier the trading endpoints cannot be used at all. It is kept minimal and marked as supporting.

1.2 Environments and base URLs

ENVIRONMENT	BASE URL	BEHAVIOR
Sandbox	<code>https://api.sandbox.brokerage.example.com/v1</code>	Paper money, simulated fills against a delayed book. Functionally identical surface. Use it for development, conformance, and CI.
Production	<code>https://api.brokerage.example.com/v1</code>	Live orders against real money and a real customer's account.
Authorization	<code>https://auth.brokerage.example.com</code>	OAuth 2.1 authorization server. Separate host from the resource API by design.

RECOMMENDATION

Make the base URL a deployment configuration value with **sandbox as the default**, so that reaching production is an explicit act rather than something a missing environment variable can cause. The example clients in Section 6 are written this way.

1.3 The design contract at a glance

CONCERN	DECISION
Style	Resource-oriented REST over HTTPS with JSON representations. Nouns as paths, HTTP methods as verbs, status codes as outcomes.
Hypermedia	Every representation carries a <code>_links</code> map. Clients bookmark the API root and discover everything else (§3.1).
Authorization	OAuth 2.1 authorization code grant with mandatory PKCE (S256), optionally DPoP-bound (§2).
Errors	RFC 9457 <code>application/problem+json</code> , with a stable, dereferenceable <code>type</code> URI to branch on (§3.2).
Money and quantities	Decimal strings plus an ISO 4217 currency. Never IEEE-754 floats (§3.3).
Time	RFC 3339 timestamps, always UTC, always with an explicit offset.
Pagination	Opaque forward cursors, surfaced as a <code>next</code> link rather than a page number (§3.4).
Safe retries	<code>Idempotency-Key</code> required on every unsafe, non-idempotent operation (§3.5).
Concurrency	Strong <code>ETag</code> plus mandatory <code>If-Match</code> on order mutation (§3.6).
Rate limits	RFC 9913 <code>RateLimit</code> headers; <code>Retry-After</code> on 429 (§3.7).

CONCERN	DECISION
Versioning	Major version in the path (/v1). Additive changes ship without a bump, so clients must tolerate unknown fields and enum values (§3.8).

1.4 Reference implementations we borrowed from

Two production brokerage APIs informed this design. Where their choices are good, we followed them; where they reflect history rather than intent, we did not.

SOURCE	WHAT WE ADOPTED	WHAT WE CHANGED
Webull developer.webull.com	Clear separation of trading, market data, and account concerns. Explicit open/close semantics on option orders. A dedicated preview step before submission.	Webull authenticates with an app key and HMAC request signing. We use OAuth 2.1 instead, so the customer grants scoped access without the application ever holding a long-lived shared secret.
Interactive Brokers interactivebrokers.com/docs/web-api	Rich order types, multi-leg combination orders priced as a unit, and a portfolio model that separates ledger, positions, and summary.	IBKR relies on a stateful session that must be kept alive by periodic /tick- le calls, and on OAuth 1.0a for third parties. We use stateless bearer tokens: no session to keep warm, no request signing ceremony.

ON THE OAUTH CHOICE

Neither reference API uses OAuth 2.1 with PKCE today. Adopting it here is a deliberate departure, and Section 2.1 makes the argument for it. The short version: request signing forces every integrator to hold a credential that can do everything forever, while a scoped, expiring, revocable token lets a customer grant read-only access to one application and full trading access to another — and take either back without changing a secret.

2 Authentication and Authorization

Every request to this API carries an OAuth 2.1 access token obtained through the authorization code flow with PKCE. This section explains how to get one, how to keep it fresh, and what each moving part is actually defending against.

2.1 Why OAuth 2.1 with PKCE

Neither reference brokerage uses this scheme. Webull signs requests with an app key and secret; Interactive Brokers maintains a stateful session and offers OAuth 1.0a to third parties. Both work. Both also put the integrator in possession of a credential that is long-lived, unscoped, and awkward to revoke without breaking every deployment at once.

OAuth 2.1 changes three things that matter in a brokerage context:

PROPERTY	SHARED-SECRET SIGNING	OAUTH 2.1 BEARER TOKEN
Blast radius	One secret, full capability, indefinite lifetime. Leak it and the attacker can trade until someone notices.	15-minute access token, limited to granted scopes. Leaked read scope cannot place an order.
Consent	Implicit. Whoever holds the secret has whatever the account has.	Explicit and per-application. The customer sees exactly which permissions they are granting, and to whom.
Revocation	Rotate the secret, redeploy everything that used it.	Revoke one grant from a settings screen. Nothing else is affected.

OAuth 2.1 is a consolidation of OAuth 2.0 plus a decade of security best current practice. For our purposes the load-bearing changes are:

- **PKCE is mandatory for all clients**, not just public ones. Previously PKCE was a mobile-app mitigation; 2.1 makes it universal because authorization codes leak through more channels than anyone originally modeled — referrer headers, browser history, OS-level URL handlers, and logging middleware.
- **The implicit grant is removed**. It returned an access token directly in a URL fragment, where it landed in history and logs. There is no supported way to obtain a token without a back-channel exchange.
- **The resource owner password credentials grant is removed**. It required the application to handle the customer's actual brokerage password. We will never ask for one, and neither should you.
- **Redirect URIs must match exactly**, by string comparison. No wildcards, no prefix matching — those were reliably exploitable for code interception.
- **Bearer tokens must not travel in query strings**. Always the `Authorization` header.

NEVER DO THIS

Do not build a screen that collects a customer's brokerage username and password and replays them. That pattern — credential sharing — is what OAuth exists to eliminate. It defeats multi-factor authentication, makes revocation impossible, and puts your application in scope for a class of liability it does not want.

2.2 Client registration and client types

Registration yields a `client_id` and a set of exactly-matched redirect URIs. Whether you also receive a credential depends on what you are building.

CLIENT TYPE	EXAMPLES	AUTHENTICATION AT THE TOKEN ENDPOINT
Public	Single-page apps, mobile apps, desktop apps, CLI tools	None. PKCE alone. A secret shipped inside a binary or a JavaScript bundle is not a secret — it only creates the illusion of one.
Confidential	Server-side web applications, backend services	<code>private_key_jwt</code> — you sign a short-lived assertion with a private key whose public half we hold. Preferred over a shared secret because the credential never crosses the wire.

Confidential clients still use PKCE. The two protect different links in the chain: client authentication proves *which application* is redeeming the code, while PKCE proves it is the *same instance* that started the flow.

2.3 The authorization code flow, end to end

The diagram below is the whole protocol. Nothing else in this section introduces a step that is not on it.

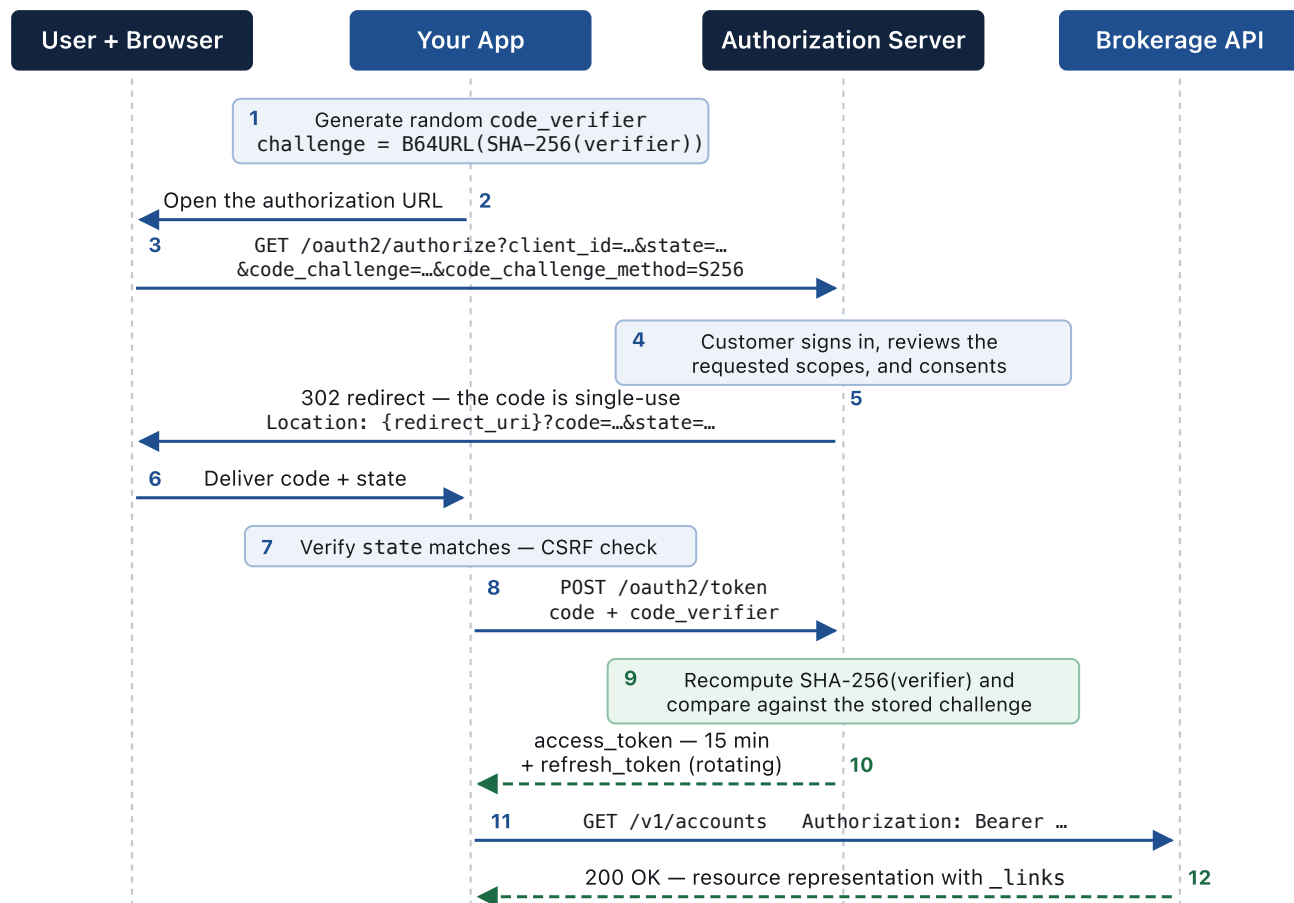


Figure 2.1 — Authorization code flow with PKCE. Steps 1, 7, and 9 are what PKCE adds to a plain authorization code exchange. The authorization code in step 5 travels through the browser and is therefore assumed to be observable; it is useless without the `code_verifier`, which never leaves your application until step 8's back-channel POST.

2.4 PKCE mechanics: verifier, challenge, and what it defends

PKCE (RFC 7636, pronounced "pixy") binds an authorization code to the specific client instance that requested it.

GENERATING THE PAIR

```
# code_verifier — 43 to 128 characters from [A-Z a-z 0-9 - . _ ~]
# 32 random bytes, base64url-encoded without padding, yields exactly 43.
code_verifier = BASE64URL_NOPAD(random_bytes(32))
code_challenge = BASE64URL_NOPAD(SHA256(ASCII(code_verifier)))
code_challenge_method = "S256"
```

REQUIREMENT	RULE
Entropy source	A cryptographically secure RNG. <code>SecureRandom</code> , <code>secrets</code> , <code>crypto.randomBytes</code> — never <code>Math.random()</code> or a seeded PRNG.
Challenge method	<code>S256</code> only. The <code>plain</code> method sends the verifier itself as the challenge, which defends against nothing. This server rejects it.
Lifetime	One verifier per authorization attempt. Never reuse one, and discard it once the code is redeemed.
Storage	Held only in the client instance that started the flow — session storage for a web app, memory for a native app. It is never transmitted until step 8.

PKCE AND `STATE` SOLVE DIFFERENT PROBLEMS

They are frequently confused, and sending only one leaves a real hole. Send both.

PARAMETER	ATTACK IT PREVENTS	HOW
<code>state</code>	CSRF on the redirect. An attacker tricks the victim's browser into completing a flow that binds the attacker's account to the victim's session.	An unguessable value generated by the client, echoed back by the server, and compared before the code is used.
<code>code_verifier</code>	Authorization code interception. A malicious app registers the same custom URL scheme, or the code leaks via a referrer, log, or browser history.	The code alone cannot be redeemed. The token endpoint requires the pre-image of the challenge, which the interceptor never saw.

COMPARE IN CONSTANT TIME

Verify `state` with a constant-time comparison (`MessageDigest.isEqual`, `secrets.compare_digest`, `crypto.timingSafeEqual`), not `==`. The timing leak is small, but the habit is what matters — the same code path tends to get reused for comparisons where it is not small.

2.5 Exchanging the code for a bearer token

POST `https://auth.brokerage.example.com/oauth2/token`

Content-Type: `application/x-www-form-urlencoded` · No authorization header for public clients

```
POST /oauth2/token HTTP/1.1
Host: auth.brokerage.example.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=Sp1xl0BeZQQYbYS6WxSbIA
&redirect_uri=https%3A%2F%2Fapp.example.com%2Fcallback
&client_id=your-client-id
&code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWF0EjXk
```

Success — 200 OK

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsImtpZCI6IjIwMjYtMDgifQ...",
  "token_type": "Bearer",
  "expires_in": 900,
  "refresh_token": "v1.MTo3ZjNhOWIyYy0...",
  "scope": "accounts:read positions:read orders:read orders:write offline_access"
}
```

READ THE GRANTED SCOPE, DO NOT ASSUME IT

The `scope` in the response is authoritative and may be **narrower** than what you requested — the customer can decline individual permissions on the consent screen. Drive your UI from the granted scope. An application that hides its trade button when `orders:write` was not granted is far better than one that shows it and surfaces a **403** later.

Failure — 400 Bad Request

```
{ "error": "invalid_grant",
  "error_description": "Authorization code expired, already used, or PKCE verification failed." }
```

ERROR	USUAL CAUSE
<code>invalid_grant</code>	Code expired (60 seconds), already redeemed, or the <code>code_verifier</code> does not hash to the challenge. Codes are strictly single-use.
<code>invalid_client</code>	Unknown <code>client_id</code> , or a confidential client failed authentication.
<code>invalid_request</code>	Missing <code>code_verifier</code> , or a <code>redirect_uri</code> that does not exactly match the one used in the authorize request.
<code>unauthorized_client</code>	The client is not permitted to use this grant type.

2.6 Access tokens: structure, lifetime, and validation

Access tokens are RS256-signed JWTs valid for **15 minutes**. Present one on every API request:

```
GET /v1/accounts HTTP/1.1
Host: api.brokerage.example.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIsImtpZCI6IjIwMjYtMDgifQ...
Accept: application/json
```

CLAIMS

```
{
  "iss": "https://auth.brokerage.example.com", // issuer
  "sub": "user_01HQ8Z1A2B3C4D5E",           // the customer
  "aud": "https://api.brokerage.example.com", // intended resource server
  "exp": 1755443402,                         // expiry, epoch seconds
  "iat": 1755442502,
  "jti": "01HQ9D4N7Q1S5U8X9Z2A4C6E",        // unique id, for revocation lists
  "scope": "accounts:read orders:read orders:write",
  "client_id": "your-client-id",
  "accounts": ["acct_01HQ8Z4M2X9K3P7R"]      // accounts in scope for this grant
}
```

IF YOU VALIDATE TOKENS YOURSELF

A resource server verifying these directly must check the signature against the JWKS at `/.well-known/jwks.json` (cached, keyed by `kid`), and then verify `iss`, `aud`, and `exp`. Skipping the `aud` check is the classic mistake: it lets a token minted for a different resource server be replayed against yours.

A **client** should not parse the token at all. Treat it as opaque, send it, and handle **401**.

WHY 15 MINUTES

Short enough that a leaked token has a small window; long enough that a normal session does not spend its time refreshing. The pairing matters more than the number: a short access token plus a rotating refresh token gives you both a small exposure window and a session that survives without re-prompting the customer.

2.7 Refresh tokens, rotation, and reuse detection

POST `https://auth.brokerage.example.com/oauth2/token`

Requires the `offline_access` scope to have been granted

```
grant_type=refresh_token
&refresh_token=v1.MTo3ZjNhOWIyYy0...
&client_id=your-client-id
```

Refresh tokens are opaque, valid for **90 days of inactivity**, and **rotate on every use**. The response always contains a new `refresh_token`; the one you sent is dead the instant the response is generated.

NORMAL ROTATION



REUSE DETECTION

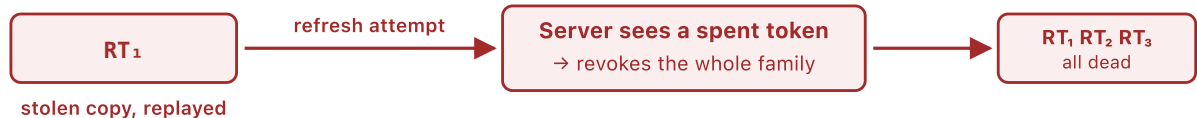


Figure 2.2 — Refresh token rotation and reuse detection. Rotation turns a stolen refresh token from an indefinite foothold into a race the attacker usually loses. Whoever refreshes second presents a token the server has already seen spent, and that presentation is treated as proof of compromise: the entire family is revoked and the customer re-authORIZES.

PERSIST THE NEW TOKEN BEFORE YOU USE THE NEW ACCESS TOKEN

Rotation makes the write ordering safety-critical. If your process crashes after receiving **RT₃** but before durably storing it, you hold a dead **RT₂** and the customer must re-authorize. Write the new refresh token to durable storage **first** — atomically, via write-to-temp-then-rename or a transaction — and only then proceed. The example clients in Section 6 all do this.

CONCURRENCY

Two threads refreshing the same token simultaneously will trip reuse detection and revoke the family. Serialize refresh behind a mutex, and have the losing caller wait for and reuse the winner's result rather than issuing its own request. This is the single most common production bug in OAuth clients with rotation enabled.

2.8 Scopes and least privilege

SCOPE	GRANTS
accounts:read	The account list and account metadata. Account numbers are masked.
accounts:read:full	Unmasked account numbers. Subject to additional review — request it only if you have a concrete need, such as ACAT transfer initiation.
positions:read	Open positions, cost basis, and unrealized P&L.
orders:read	Order status, order history, executions, and the event stream.
orders:write	Place, amend, and cancel stock and option orders. The consequential one.
transactions:read	Transaction and settlement history.

SCOPE	GRANTS
<code>instruments:read</code>	Instrument search and option chains.
<code>offline_access</code>	Issues a refresh token. Without it, access ends when the access token expires.

ASK FOR LESS

Request the narrowest set that makes your application work, and request additional scopes later through a fresh authorization when the customer reaches a feature that needs them. A portfolio dashboard that asks for `orders:write` on first launch reads as a red flag, and consent rates reflect that.

2.9 Sender-constrained tokens with DPoP

A bearer token is, definitionally, usable by whoever bears it. DPoP (RFC 9449) binds a token to a key pair your client holds, so a stolen token is inert without the private key.

Each request carries a `DPoP` proof JWT, signed with your private key, covering the HTTP method and URI plus a unique nonce and timestamp:

```
GET /v1/accounts/acct_01HQ8Z4M2X9K3P7R/orders HTTP/1.1
Authorization: DPoP eyJhbGciOiJSUzI1NiIsInR5cCI6ImRwb3Arand0Ii...
DPoP: eyJ0eXAiOiJkcG9wK2p3dCIsImFsZyI6IktVMjU2IiwiaWandrIjp7...
```

DPoP is **required** for `orders:write` on accounts above the institutional risk threshold, and recommended for every integration that can manage a key pair. Request it by including a `dpop_jkt` parameter on the authorization request; the issued token then carries a `cnf` confirmation claim and will be rejected if presented as a plain bearer token.

2.10 Revocation, logout, and token hygiene

POST <https://auth.brokerage.example.com/oauth2/revoke>

```
token=v1.MTo3ZjNh0WIyYy0...&token_type_hint=refresh_token&client_id=your-client-id
```

Revoking a refresh token invalidates the whole family, including outstanding access tokens. Returns **200 OK** whether or not the token existed — per RFC 7009, so the endpoint cannot be used as a token oracle.

PRACTICE	REASON
Revoke on logout	Dropping tokens on the floor leaves a live account credential in whatever logs, crash dumps, or backups happened to capture it.
Never log tokens	Redact <code>Authorization</code> headers and token responses in every logger, APM agent, and error reporter. This is the most common way tokens leak in practice.
Encrypt at rest	Keychain or Keystore on device; a managed secret store on a server. File storage, if unavoidable, must be owner-only (mode <code>0600</code>).
Never in URLs	Query strings land in access logs, referrer headers, and browser history. Header only.
One grant per user per app	Do not share a token across customers or pool credentials. The <code>sub</code> claim is what makes an audit trail meaningful.

2.11 Device authorization for input-constrained clients

Where a redirect-based browser flow is not possible — a TV app, a headless terminal, an embedded device — use the device authorization grant (RFC 8628):

```

1. POST /oauth2/device_authorization → device_code, user_code, verification_uri, interval
2. Display the user_code and verification_uri to the customer
3. Poll POST /oauth2/token with grant_type=urn:ietf:params:oauth:grant-type:device_code
   // authorization_pending → keep polling at `interval`
   // slow_down             → increase the interval by 5s and continue
   // access_denied          → the customer declined; stop
4. On success: access_token + refresh_token, exactly as in §2.5

```

SCOPE RESTRICTION

The device flow does **not** grant `orders:write`. Its user code is short and typed by hand, which makes it a weaker binding than a full redirect flow — acceptable for read access, not for placing trades.

3 API Conventions

These rules apply to every endpoint. Reading this section once will save you from rediscovering the same eight things across five sections of reference material.

3.1 Hypermedia and HATEOAS

Every representation carries a `_links` object: a map keyed by link relation, each value carrying at least an `href`.

```
{
  "orderId": "ord_01HQ9D4N7Q1S5U8X",
  "status": "PARTIALLY_FILLED",
  "_links": {
    "self": { "href": "https://api.brokerage.example.com/v1/accounts/acct_.../orders/ord_...",
    "cancel": { "href": ".../orders/ord_...", "method": "DELETE", "title": "Cancel the unfilled 200
shares" },
    "replace": { "href": ".../orders/ord_...", "method": "PATCH" },
    "executions": { "href": ".../orders/ord_.../executions" }
  }
}
```

The contract has three parts, and the third is the one that pays for the other two:

1. **One bookmarked URL.** Clients hard-code `GET /` and discover everything else by following relations.
2. **No URL construction.** Never build a URL by concatenating strings. Follow the `href` you were given, verbatim — including its query parameters.
3. **Links are state-dependent.** A `FILLED` order has no `cancel` link. An account without options approval has no `place-option-order` link. **Branch on the presence of a link, not on a status string.**

WHY POINT 3 MATTERS MORE THAN IT LOOKS

Status enumerations grow. If your UI enables its Cancel button when `status ∈ {NEW, PARTIALLY_FILLED}`, then the day a `PENDING_ROUTE` status ships, your button silently disappears for orders that are in fact cancellable. If instead it enables on `"cancel" in _links`, it keeps working — the server already knows the answer, and it is telling you.

RELATION	MEANING
self	Canonical URL of this resource.
next	The next page. Absent on the last page (§3.4).
account , order , instrument	Navigate to a related resource.
balances , positions , orders , transactions , executions	Sub-collections of an account or order.
place-order , cancel , replace , close-position	State-changing affordances. Each carries an explicit method .
events	The server-sent event stream for this collection.

3.2 Errors: RFC 9457 problem details

Every error response uses `application/problem+json` :

```

HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://developer.brokerage.example.com/problems/insufficient-buying-power",
  "title": "Insufficient buying power",
  "status": 422,
  "detail": "Order requires 123,750.00 USD of stock buying power; 84,360.20 USD available.",
  "instance": "/v1/accounts/acct_01HQ8Z4M2X9K3P7R/orders",
  "required": { "amount": "123750.00", "currency": "USD" },
  "available": { "amount": "84360.20", "currency": "USD" },
  "traceId": "01HQ9D5P8R2T6V9Y0B3D5F7H"
}
```

FIELD	HOW TO TREAT IT
<code>type</code>	The only field safe to branch on. A stable, dereferenceable URI identifying the problem class. It will not change without a version bump.
<code>title</code> , <code>detail</code>	Human-facing prose. Reworded without notice, and localized in future. Display it; never parse it.
<code>instance</code>	Which request this was about.
<code>traceId</code>	Log this on every failure. It is the first thing support will ask for and the only way to find the request in our logs.
<code>errors[]</code>	Field-level failures on validation problems, each with an RFC 6901 JSON Pointer.
Extensions	Problem types add their own fields — <code>required</code> / <code>available</code> here, <code>requiredLevel</code> on an options-approval failure. Documented per type in Appendix A.

STATUS CODE SEMANTICS

STATUS	MEANING IN THIS API
400	Malformed syntax or schema validation failure. The request never reached business logic.
401	Token missing, expired, malformed, or revoked. Refresh once and retry; if that fails, re-authorize.
403	Authenticated but not permitted — missing scope, or an options strategy above the account's approval level.
404	No such resource, or it exists outside your grant. Deliberately indistinguishable, so the API cannot be used to enumerate account identifiers.
409	Conflicting state — the account is restricted, or the order already reached a terminal state.
412	<code>If-Match</code> did not match. The resource changed after you read it (§3.6).
422	Well-formed but unacceptable — insufficient buying power, price outside collar, market closed, untradable instrument.
428	A required precondition header was not supplied.
429	Rate limited. Honor <code>Retry-After</code> (§3.7).
503	Upstream venue or clearing system unavailable. Retryable with backoff.

400 VERSUS 422 IS NOT COSMETIC

A **400** means your code is wrong — fix it and the request will work. A **422** means your code is fine and the *world* said no: not enough buying power, market closed, instrument halted. The first is a bug to fix; the second is a condition to surface to the customer. Treating them identically produces an application that shows "Invalid request" when the honest answer is "the market is closed".

3.3 Decimals, money, and why nothing is a float

Prices, quantities, and monetary amounts are transmitted as JSON **strings**:

```
{ "limitPrice": "412.50", "quantity": "300", "netLiquidation": "184320.55" }
```

This is not stylistic. JSON numbers are IEEE-754 doubles in most parsers, and a double cannot represent `0.1` exactly. In a domain where a share count and a limit price multiply into a cash obligation, that error is a defect with a dollar value attached.

LANGUAGE	USE	NEVER
Java	<code>new BigDecimal("412.50")</code>	<code>double</code> , <code>Double.parseDouble</code> , or <code>BigDecimal.valueOf(double)</code>
Python	<code>Decimal("412.50")</code>	<code>float</code> , or <code>Decimal(0.1)</code> from a float literal
JavaScript	<code>decimal.js</code> / <code>big.js</code> , or keep the string	<code>Number()</code> , <code>parseFloat</code> , or arithmetic on either
Go	<code>shopspring/decimal</code>	<code>float64</code>

THE SUBTLE JAVA TRAP

`BigDecimal.valueOf(0.1)` and `new BigDecimal(0.1)` both route through a `double` and inherit its error before `BigDecimal` ever sees the value. Only the **String** constructor is safe. The same applies to Python's `Decimal(0.1)` versus `Decimal("0.1")` .

Other formats follow the same principle of being unambiguous on the wire:

Timestamps

RFC 3339, UTC, explicit offset: `2026-08-17T14:31:02Z` . Sub-second precision on executions.

Dates

ISO 8601 calendar dates for trade and settlement dates: `2026-08-19` . These are exchange-local dates, not instants.

Currency

ISO 4217 alphabetic codes. Every monetary field either carries a `currency` or sits beside one.

Identifiers

Opaque, prefixed, sortable strings (`acct_` , `ord_` , `pos_` , `txn_` , `exec_` , `inst_`). Do not parse them; the prefix is a debugging affordance, not an API.

3.4 Pagination

Collections use opaque forward cursors. Request a page size with `limit` (1–200, default 50); follow the `next` link for more.

```
GET /v1/accounts/acct_.../transactions?from=2026-08-01&limit=50

{ "items": [ ... ],
  "_links": {
    "self": { "href": ".../transactions?from=2026-08-01&limit=50" },
    "next": { "href": ".../transactions?from=2026-08-01&limit=50&cursor=eyJvIjoxMjAwfQ" }
  } }
```

No `next` link means you have reached the end. Cursors are opaque: their encoding is not part of the contract and will change. Do not construct, parse, or persist them.

WHY CURSORS RATHER THAN OFFSETS

Offset pagination is unstable against a moving dataset. Between page 1 and page 2 of your transaction history, a settlement can insert a back-dated row — and with `OFFSET 50` you silently skip a record. Cursors are anchored to a position in a stable ordering, so insertions do not shift the window out from under you.

Rows can still legitimately repeat across pages when settlement back-dates an entry. **Reconcile on** `transactionId`, which is permanent and unique.

3.5 Idempotency

Every unsafe, non-idempotent operation — in practice, placing and amending orders — requires an `Idempotency-Key` header carrying a client-generated UUIDv4.

```
POST /v1/accounts/acct_.../orders
Idempotency-Key: 6f1c2a90-3f5b-4c1e-9c2a-8d7b6e5f4a30
```

SCENARIO	RESULT
First request with a key	201 The order is created.
Replay, same key, identical body	200 The original order is returned with <code>Idempotency-Replayed: true</code> . No second order.
Replay, same key, different body	422 <code>idempotency-key-reuse</code> . Nothing is created.
Key older than 24 hours	Treated as new. Keys are retained for 24 hours.

GENERATE THE KEY ONCE, PER INTENT

The key must be created **before the first attempt** and reused verbatim on every retry of that same order intent. Generating a fresh UUID inside a retry loop defeats the entire mechanism and is the exact bug it exists to prevent.

The failure being defended against is specific: a socket timeout on a market order is indistinguishable, from the client's side, from a rejection. The order may have been accepted and routed. Retrying without a key doubles the customer's position; retrying with one is safe.

3.6 Optimistic concurrency with ETags

Order reads return a strong `ETag`. Order mutations — `PATCH` and `DELETE` — require `If-Match`.

```
# 1. Read, and keep the validator
GET /v1/accounts/acct_.../orders/ord_...
→ 200 OK
   ETag: W/"ord-7-1755441062"

# 2. Mutate, asserting you saw that exact version
PATCH /v1/accounts/acct_.../orders/ord_...
If-Match: W/"ord-7-1755441062"
Content-Type: application/merge-patch+json

{ "limitPrice": "413.25" }
```

OUTCOME	MEANING
200	The order was unchanged since your read. The amendment was accepted, and a new <code>ETag</code> is returned.
412	The order changed after your read. Re-read it and decide again — do not blindly retry with the newer <code>ETag</code> , or you will amend an order whose state you never evaluated.
428	You omitted <code>If-Match</code> . Rejected rather than defaulted, on purpose.

THE RACE THIS CLOSES

You read an order showing 100 of 300 shares filled and decide to reprice the remaining 200. While you are deciding, it fills completely. Without `If-Match`, your amendment lands against an order you believe is working, and the outcome depends on timing you cannot observe. With it, you get a `412` and a chance to look again.

`ETag` also works for cheap polling: send it as `If-None-Match` on a `GET` and receive `304 Not Modified` with no body when nothing has changed. That is what keeps a one-second poll inside the rate limit.

3.7 Rate limits and backoff

Every response carries the current quota state, per RFC 9913:

```
RateLimit: "default";r=94;t=51      # 94 requests remain, window resets in 51s
RateLimit-Policy: "default";q=120;w=60 # 120 requests per 60-second window
```

POLICY	QUOTA	APPLIES TO
default	120 / minute	All read endpoints.
orders-write	60 / minute	Place, amend, cancel.
order-poll	1 / second per order	GET on a single order. Use the event stream instead.
streams	5 concurrent	Open SSE connections per grant.

On `429`, wait `Retry-After` seconds and retry with **full jitter** — a random delay between zero and your computed cap, rather than a fixed backoff:

```
delay = random_between(0, min(2^attempt × base, cap))
```

WHY JITTER, SPECIFICALLY

When a venue hiccups, every client fails at once and every client retries at once. Fixed backoff preserves that synchronization and reproduces the thundering herd on each round, turning a two-second blip into a sustained outage. Randomizing the delay spreads the retries out. Ignoring `Retry-After` repeatedly escalates to a temporary suspension of the client.

3.8 Versioning and compatibility

The major version is pinned in the server path (`/v1`). Within a major version, the following are considered **backward-compatible** and ship without notice:

- New endpoints, and new optional request fields.
- New fields in a response object.
- New members in an enumeration — including new `OrderStatus` values.
- New link relations in `_links`.
- New problem `type` URIs.

Which places two obligations on your client:

1. **Ignore unknown properties.** Configure your deserializer to tolerate them — Jackson's `FAIL_ON_UNKNOWN_PROPERTIES=false`, or an equivalent. A strict parser will break on a purely additive change.
2. **Tolerate unknown enum values.** Do not map to a closed enum that throws. Treat an unrecognized `OrderStatus` as working, and rely on `_links` to know what you may do with it.

Breaking changes — removing a field, changing a type, tightening validation — require a new major version. The previous version is supported for at least 12 months after the successor ships.

4 Accounts APIs

Five read-only views of a customer's brokerage accounts: which accounts you can reach, what they are worth, what they hold, what is working, and what has already happened.

4.1 Account list

GET `/v1/accounts`

Requires `accounts:read`

Returns every account the presented access token is authorized to see. This is the entry point for everything else — the `accountId` returned here is the opaque, permanent handle used in every other path.

```
GET /v1/accounts HTTP/1.1
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...

→ 200 OK
{
  "items": [
    {
      "accountId": "acct_01HQ8Z4M2X9K3P7R",
      "accountNumber": "....4471",
      "nickname": "Main brokerage",
      "type": "MARGIN",
      "registration": "INDIVIDUAL",
      "status": "ACTIVE",
      "baseCurrency": "USD",
      "optionsLevel": "LEVEL_3",
      "dayTrader": true,
      "openedAt": "2021-03-14T00:00:00Z",
      "_links": {
        "self": { "href": ".../v1/accounts/acct_01HQ8Z4M2X9K3P7R" },
        "balances": { "href": ".../balances" },
        "positions": { "href": ".../positions" },
        "orders": { "href": ".../orders" },
        "transactions": { "href": ".../transactions" },
        "place-order": { "href": ".../orders", "method": "POST" }
      }
    }
  ],
  "_links": { "self": { "href": ".../v1/accounts" } }
}
```

FIELD	NOTES
accountId	Opaque and permanent. Use this as your foreign key.
accountNumber	Customer-facing, masked to the last four digits unless the grant carries <code>accounts: read: full</code> . Display it; do not key on it.
type	<code>CASH</code> settles trade-by-trade and cannot borrow. <code>MARGIN</code> can, which changes buying power, short-sale eligibility, and available option strategies.
status	<code>ACTIVE</code> , <code>RESTRICTED</code> , <code>CLOSED</code> , <code>PENDING</code> . A <code>RESTRICTED</code> account can be read but not traded — it carries no <code>place-order</code> link.
optionsLevel	Approved strategies, cumulative. See §5.4.
dayTrader	Pattern-day-trader flag. When true, the 25,000 USD minimum equity rule applies and day trades are counted.

The list is deliberately unpaginated — a customer rarely has more than a handful of accounts, and forcing pagination onto a bounded collection buys nothing.

4.2 Account balances

GET `/v1/accounts/{accountId}/balances`

Requires `accounts: read`

A balance **snapshot**, stamped with `asOf`. Intraday it reflects real-time marks on open positions and unsettled trade activity; after the close it settles to end-of-day values.

```
→ 200 OK
{
  "accountId":      "acct_01HQ8Z4M2X9K3P7R",
  "currency":       "USD",
  "asOf":           "2026-08-17T18:42:11Z",
  "netLiquidation": "184320.55",
  "equity":         "184320.55",
  "positionsMarketValue": "142140.45",
  "cash": {
    "total":        "42180.10",
    "settled":      "39900.00",
    "unsettled":    "2280.10",
    "withdrawable": "39900.00"
  },
  "buyingPower": {
    "stock":        "84360.20",
    "option":       "39900.00",
    "dayTrading":   "168720.40"
  },
  "margin": {
```

```

    "maintenanceRequirement": "38504.00",
    "initialRequirement":     "71140.22",
    "marginBalance":          "0.00",
    "excessLiquidity":        "145816.55",
    "marginCall":             null
  },
  "dayTradesUsed":           1,
  "dayTradesRemaining":      3,
  "patternDayTrader":        true,
  "_links": { ... }
}

```

BUYING POWER IS THREE DIFFERENT NUMBERS

They are broken out because they genuinely differ. In a margin account, `stock` is typically twice `option` — options cannot be bought on margin — and `dayTrading` is a larger figure that resets each session. **Sizing an option order against `buyingPower.stock` will get it rejected**, and the rejection will look mysterious if you were reading the wrong field.

FIELD	MEANING
<code>netLiquidation</code>	Total account value if every position were liquidated at current marks. The headline number.
<code>cash.settled</code>	Cash that has completed settlement. In a cash account , spending unsettled proceeds causes a good-faith violation — this is the figure that matters there.
<code>cash.withdrawable</code>	What can actually leave the account today, after settlement and any holds.
<code>margin.excessLiquidity</code>	Cushion above the maintenance requirement. When this approaches zero, a margin call is imminent.
<code>margin.marginCall</code>	Non-null only when a call is outstanding, carrying <code>type</code> , <code>amount</code> , and <code>dueDate</code> . The one field worth alerting on.
<code>dayTradesRemaining</code>	Day trades left in the rolling five-business-day window before the PDT rule restricts the account.

Responses carry `Cache-Control: no-store` during market hours. Do not cache balances across a trade.

4.3 Account positions

GET `/v1/accounts/{accountId}/positions`

Requires `positions:read`

Open positions, one row per instrument and side. Filter with `instrumentType` (`EQUITY`, `ETF`, `OPTION`) or `symbol` — the latter matches option positions on that underlying too. Paginated (§3.4).

→ 200 OK – a long stock position and a short call written against it

```
{ "items": [
  {
    "positionId": "pos_01HQ9A1F4C8M2Q6T",
    "instrument": { "instrumentId": "inst_eq_AAPL", "type": "EQUITY",
      "symbol": "AAPL", "currency": "USD" },
    "side": "LONG",
    "quantity": "500",
    "contractMultiplier": 1,
    "averageCost": "178.4200",
    "costBasis": "89210.00",
    "lastPrice": "231.1500",
    "marketValue": "115575.00",
    "unrealizedPnl": "26365.00",
    "unrealizedPnlPercent": "29.5539",
    "_links": { "self": {...}, "instrument": {...},
      "close-position": { "href": ".../orders", "method": "POST" } }
  },
  {
    "positionId": "pos_01HQ9A3H7D1N4S8V",
    "instrument": {
      "instrumentId": "inst_op_AAPL_20261218C00250000",
      "type": "OPTION", "symbol": "AAPL 261218C00250000", "currency": "USD",
      "option": { "underlyingSymbol": "AAPL", "expiration": "2026-12-18",
        "strike": "250.00", "optionType": "CALL",
        "exerciseStyle": "AMERICAN", "contractMultiplier": 100 }
    },
    "side": "SHORT",
    "quantity": "-5", // contracts, not shares
    "contractMultiplier": 100,
    "averageCost": "12.4000",
    "marketValue": "-4925.00",
    "unrealizedPnl": "1275.00"
  }
] }
```

OPTIONS ARE COUNTED IN CONTRACTS

A quantity of -5 on an option position is five short contracts, which is exposure to **500 shares**. Multiply by `contractMultiplier` (normally 100; adjusted contracts differ) to get share-equivalent exposure. Treating the two as the same unit understates option risk by two orders of magnitude.

`quantity` is signed — negative for short positions — so a naive sum across a portfolio nets long against short correctly. Long and short exposure in the same underlying are returned as separate positions. Closed positions do not appear here; use the transaction history for realized activity.

4.4 Account order status

GET `/v1/accounts/{accountId}/orders`

Requires `orders:read`

Orders on the account, newest first. **With no filter, this returns working orders only** — `PENDING_NEW`, `NEW`, `PARTIALLY_FILLED`, `PENDING_REPLACE`, `PENDING_CANCEL` — which is the set a trading screen needs on load. Pass `status` explicitly, or the literal `status=ALL`, to widen it.

PARAMETER	PURPOSE
<code>status</code>	Repeatable. One or more <code>OrderStatus</code> values, or <code>ALL</code> to disable the working-orders default.
<code>symbol</code>	One underlying, including its option orders.
<code>from</code> , <code>to</code>	Bounds on <code>createdAt</code> . Defaults to the last 7 days.
<code>cursor</code> , <code>limit</code>	Pagination (§3.4).

```
GET /v1/accounts/acct_.../orders?status=ALL&symbol=MSFT&limit=25
```

```
→ 200 OK
```

```
{ "items": [ {
  "orderId":           "ord_01HQ9D4N7Q1S5U8X",
  "clientOrderId":     "ui-2026-08-17-0007",
  "status":            "PARTIALLY_FILLED",
  "assetClass":        "EQUITY",
  "side":              "BUY",
  "instrument":         { "symbol": "MSFT", ... },
  "quantity":          "300",
  "filledQuantity":     "100",
  "remainingQuantity": "200",
  "orderType":         "LIMIT",
  "limitPrice":         "412.50",
  "averageFillPrice":   "412.48",
  "timeInForce":        "DAY",
  "createdAt":          "2026-08-17T14:31:02Z",
  "updatedAt":          "2026-08-17T14:33:19Z",
  "_links": {
    "self":      { "href": ".../orders/ord_01HQ9D4N7Q1S5U8X" },
    "cancel":    { "href": "...", "method": "DELETE" },
    "replace":   { "href": "...", "method": "PATCH" },
    "executions": { "href": ".../executions" }
  }
} ] }
```

SINGLE ORDER, AND ITS FILLS

GET `/v1/accounts/{accountId}/orders/{orderId}`

GET `/v1/accounts/{accountId}/orders/{orderId}/executions`

Requires `orders:read` · single-order reads return a strong `ETag`

A single order can fill in many prints across several venues. `executions` is the audit trail behind `filledQuantity` and `averageFillPrice`:

```
{ "items": [
  { "executionId": "exec_01HQ9E1A2B3C4D5E", "quantity": "100", "price": "412.48",
    "venue": "XNAS", "liquidity": "REMOVED", "executedAt": "2026-08-17T14:33:19.418Z",
    "settlementDate": "2026-08-19", "commission": "0.00" },
  { "executionId": "exec_01HQ9E4F5G6H7J8K", "quantity": "200", "price": "412.50",
    "venue": "ARCX", "liquidity": "ADDED", "executedAt": "2026-08-17T14:35:02.771Z" }
] }
```

POLL CHEAPLY, OR DO NOT POLL AT ALL

Single-order reads are limited to one request per second. Send the previous `ETag` as `If-None-Match` and an unchanged order costs you a `304` with no body. Better still, use the event stream in §5.7 and stop polling entirely.

4.5 Account transaction history

GET `/v1/accounts/{accountId}/transactions`

Requires `transactions:read`

Cash and position activity: trades, dividends, interest, fees, transfers, assignments, exercises, expirations, and corporate actions. Ordered by `tradeDate` descending, then `transactionId` descending as a stable tiebreaker.

PARAMETER	PURPOSE
<code>from</code> , <code>to</code>	Inclusive bounds on trade date. Defaults to the last 90 days; accepts any range up to 7 years.
<code>type</code>	Repeatable. Restrict to one or more transaction types.
<code>symbol</code>	Activity in one symbol or its options.

```
GET /v1/accounts/acct_.../transactions?from=2026-08-01&type=TRADE&type=DIVIDEND
```

→ 200 OK

```
{ "items": [
  { "transactionId": "txn_01HQ9C2K5M8P1R4W",
    "type": "TRADE",
    "subType": "BUY",
    "description": "Bought 100 AAPL @ 231.05",
    "tradeDate": "2026-08-15",
    "settlementDate": "2026-08-18",
    "instrument": { "symbol": "AAPL", ... },
    "quantity": "100",
    "price": "231.0500",
    "grossAmount": "-23105.00", // signed: negative reduces cash
    "fees": [ { "type": "REGULATORY", "description": "SEC Section 31 fee",
      "amount": "0.00" } ],
    "netAmount": "-23105.00", // gross net of fees – the actual cash movement
    "currency": "USD",
    "relatedOrderId": "ord_01HQ9C1J3L7N0Q3V",
    "_links": { "self": {...}, "order": { "href": ".../orders/ord_01HQ9C1J3L7N0Q3V" } } },

  { "transactionId": "txn_01HQ9B8G4J2L6N9S",
    "type": "DIVIDEND", "subType": "CASH_DIVIDEND",
    "description": "AAPL cash dividend 0.26/share on 500 shares",
    "tradeDate": "2026-08-14", "grossAmount": "130.00",
    "fees": [], "netAmount": "130.00", "currency": "USD" }
] }
```

TYPE	COVERS
TRADE	Buys and sells. <code>subType</code> narrows to <code>BUY</code> , <code>SELL</code> , <code>SELL_SHORT</code> , <code>BUY_TO_COVER</code> .
DIVIDEND, INTEREST	Cash and stock dividends, credit interest.
FEE, TAX	Commissions, regulatory fees, margin interest, withholding.
DEPOSIT, WITHDRAWAL, TRANSFER, JOURNAL	Cash and position movement in and out of the account.
ASSIGNMENT, EXERCISE, EXPIRATION	Option lifecycle events. These arrive without an order — nobody placed one — so a reconciliation keyed on <code>relatedOrderId</code> will miss them.
CORPORATE_ACTION	Splits, mergers, spin-offs, symbol changes.

RECONCILE ON `TRANSACTIONID`, NOT ON POSITION IN THE LIST

Settlement can back-date an entry, which means a transaction may appear in a page you have already read.

`transactionId` is permanent and unique; deduplicate on it. Every example client in Section 6 does this rather than summing pages blindly.

Ranges wider than one year are served from cold storage and may take several seconds. Pull history incrementally and cache it locally rather than re-fetching seven years on every run.

5 Trading APIs

Placing, amending, and cancelling stock and option orders. Everything in this section requires the `orders:write` scope and an `ACTIVE` account, and everything in this section moves real money in production.

5.1 The order lifecycle

An order is a resource with a state machine. Understanding it is the difference between an integration that reconciles cleanly and one that quietly loses track of fills.

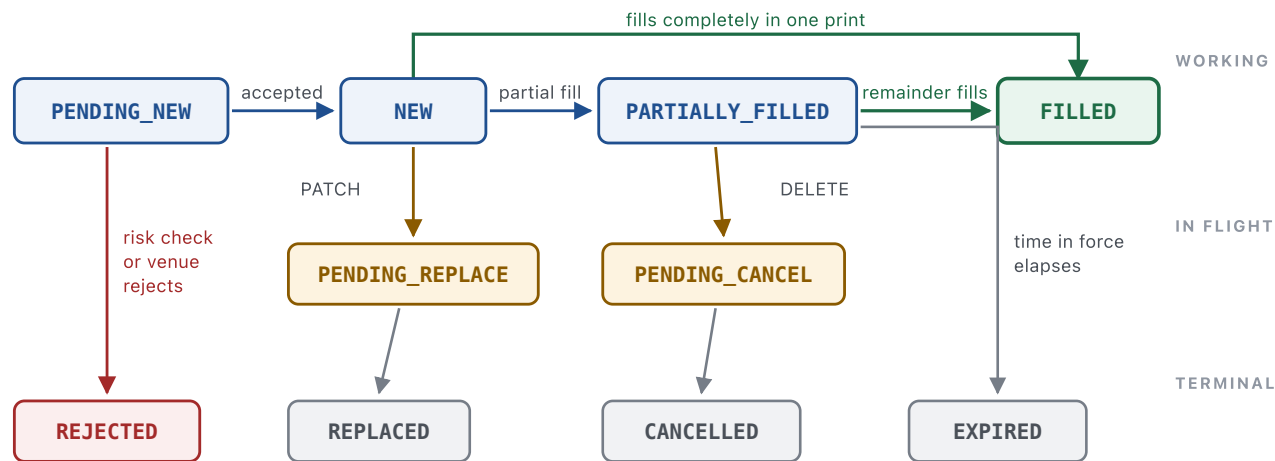


Figure 5.1 — Order lifecycle. For legibility the diagram shows `PATCH` from `NEW` and `DELETE` from `PARTIALLY_FILLED` ; both transitions are available from either working state. A `REPLACED` order is terminal and its replacement is a **new resource with a new `orderId`** — reconciliation must follow that link rather than assume the identifier is stable.

STATE	TERMINAL?	MEANING
<code>PENDING_NEW</code>	No	Accepted by us, not yet acknowledged by the venue.
<code>NEW</code>	No	Live on the book. Working, unfilled.
<code>PARTIALLY_FILLED</code>	No	Some quantity filled; the remainder is still working.
<code>PENDING_REPLACE</code> / <code>PENDING_CANCEL</code>	No	An amendment or cancellation is in flight at the venue.
<code>FILLED</code>	Yes	Fully executed.
<code>CANCELLED</code>	Yes	Cancelled. May carry a non-zero <code>filledQuantity</code> if it partially filled first.

STATE	TERMINAL?	MEANING
REJECTED	Yes	Refused. <code>rejectReason</code> carries a machine-readable <code>code</code> .
EXPIRED	Yes	Time in force elapsed — a <code>DAY</code> order at the close, or a <code>GTD</code> order past its date.
REPLACED	Yes	Superseded by an amendment. Follow the link to the replacement.
SUSPENDED	No	Held — a trading halt in the instrument, or a compliance hold on the account.

THREE THINGS THAT REGULARLY SURPRISE PEOPLE

A 201 is not a fill. It means the order was accepted and routed. Terminal state arrives later, via the event stream, a webhook, or a poll.

Cancellation is a request, not a guarantee. The order moves to `PENDING_CANCEL` and only reaches `CANCELLED` once the venue acknowledges. A fill already in flight can still land — a "cancelled" order can end up `FILLED` .

An order can be rejected after acceptance. We accept and route it; the venue refuses it a moment later. That surfaces as a later `REJECTED` state, not as an error on the original call.

5.2 Placing an order safely

POST `/v1/accounts/{accountId}/orders`

Requires `orders:write` · `Idempotency-Key` header **required**

The request body is a discriminated union on `assetClass` : `EQUITY` for single-leg stock and ETF orders, `OPTION` for single- and multi-leg listed option orders.

VALIDATION ORDER

The service validates in this sequence and **stops at the first failure**, so a rejection tells you the first thing wrong, not everything wrong:

```

schema → instrument tradability → market / session eligibility
        → options approval level → buying power and margin
        → risk limits             → route to venue

```

THE SAFE SUBMISSION PATTERN

```
// 1. Generate the idempotency key ONCE, before the first attempt.
key = uuid4()

// 2. Preview, so the customer confirms real numbers rather than an estimate.
preview = POST /orders/preview {order}
if (!preview.valid) { show preview.warnings; stop }

// 3. Submit. Retry ONLY with the same key – never regenerate it in the loop.
retry (max 4, full jitter) {
  response = POST /orders {order} Idempotency-Key: key
}

// 4. A 201 means routed, not filled. Confirm terminal state separately.
watch(response.orderId) // SSE stream, webhook, or ETag-conditional poll
```

5.3 Stock trading

FIELD	NOTES
side	BUY , SELL , SELL_SHORT , BUY_TO_COVER . Short sales require a margin account and a locate.
instrumentId	From instrument search. Not a ticker — tickers are ambiguous across venues and change on corporate actions.
quantity	Positive decimal string. Fractional quantities only where fractionable is true, and only for MARKET / DAY .
orderType	MARKET , LIMIT , STOP , STOP_LIMIT , TRAILING_STOP , TRAILING_STOP_LIMIT .
timeInForce	DAY , GTC , GTD , IOC , FOK , OPG , CLS . See Appendix B.
extendedHours	Pre- and post-market routing. Must be LIMIT and DAY ; spreads are wider and liquidity thinner.
bracket	Attached profit target and stop, released on entry fill, mutually one-cancels-the-other.

LIMIT BUY

```
POST /v1/accounts/acct_.../orders
Idempotency-Key: 6f1c2a90-3f5b-4c1e-9c2a-8d7b6e5f4a30

{ "assetClass": "EQUITY",
  "clientOrderId": "ui-2026-08-17-0007",
  "side": "BUY",
  "instrumentId": "inst_eq_MSFT",
  "quantity": "300",
  "orderType": "LIMIT",
  "limitPrice": "412.50",
  "timeInForce": "DAY",
  "extendedHours": false }

→ 201 Created
Location: /v1/accounts/acct_.../orders/ord_01HQ9D4N7Q1S5U8X
ETag: W/"ord-1-1755441062"
```

BRACKET ORDER

```
{ "assetClass": "EQUITY", "side": "BUY", "instrumentId": "inst_eq_MSFT",
  "quantity": "100", "orderType": "LIMIT", "limitPrice": "410.00",
  "timeInForce": "DAY",
  "bracket": {
    "takeProfit": { "orderType": "LIMIT", "limitPrice": "430.00" },
    "stopLoss": { "orderType": "STOP", "stopPrice": "398.00" }
  } }
```

TRAILING STOP

```
{ "assetClass": "EQUITY", "side": "SELL", "instrumentId": "inst_eq_NVDA",
  "quantity": "50", "orderType": "TRAILING_STOP",
  "trailingPercent": "5.0", // or trailingAmount - mutually exclusive
  "timeInForce": "GTC" }
```

MARKET ORDERS AND GAPS

A **MARKET** order guarantees execution, not price. On an illiquid name, at the open, or during a halt resumption, the fill can be far from the last quote. Prefer marketable limits for anything the customer would be upset to see filled 8% away, and surface the preview's estimated cost before submitting.

5.4 Options trading

Option orders carry a **legs** array — one entry for a single-leg order, up to four for a combination. Multi-leg orders are priced and routed **as one unit**; legs never fill independently.

APPROVAL LEVELS

LEVEL	PERMITS (CUMULATIVE)
LEVEL_1	Covered calls, cash-secured puts.
LEVEL_2	Long calls and puts.
LEVEL_3	Spreads and other defined-risk multi-leg strategies.
LEVEL_4	Uncovered (naked) short options.

An order above the account's level is rejected with `403 options-level-insufficient`, which carries `requiredLevel` and `currentLevel` so you can tell the customer exactly what to request.

OPEN AND CLOSE ARE EXPLICIT

`side` on an option leg is one of `BUY_TO_OPEN`, `BUY_TO_CLOSE`, `SELL_TO_OPEN`, `SELL_TO_CLOSE`. The distinction drives margin treatment and position netting and **cannot be inferred** from buy/sell alone — selling a call you hold closes a position; selling one you do not is a naked short requiring `LEVEL_4`.

SINGLE LEG

```
{ "assetClass": "OPTION",
  "strategy": "SINGLE",
  "legs": [ { "instrumentId": "inst_op_AAPL_20261218C00250000",
              "side": "BUY_TO_OPEN", "ratio": 1 } ],
  "quantity": "5",           // 5 contracts
  "orderType": "LIMIT",
  "limitPrice": "12.40",     // per contract → 12.40 × 5 × 100 = 6,200 outlay
  "priceEffect": "DEBIT",
  "timeInForce": "DAY" }
```

VERTICAL SPREAD — PRICED AS A UNIT

```
{ "assetClass": "OPTION",
  "strategy": "VERTICAL",
  "legs": [
    { "instrumentId": "inst_op_AAPL_20261218C00250000", "side": "BUY_TO_OPEN", "ratio": 1 },
    { "instrumentId": "inst_op_AAPL_20261218C00260000", "side": "SELL_TO_OPEN", "ratio": 1 }
  ],
  "quantity": "10",         // 10 SPREADS = 20 contracts
  "orderType": "NET_DEBIT",
  "netPrice": "3.15",       // always positive
  "priceEffect": "DEBIT",   // direction lives here, not in the sign
  "timeInForce": "DAY" }
```

TWO WAYS TO GET OPTION SIZING BADLY WRONG

quantity is strategy units, not contracts. A **quantity** of 10 on a two-leg vertical is ten *spreads* — twenty contracts — with each leg scaled by its **ratio**. Reading it as contracts halves your intended position, or doubles it, depending on which way you err.

Declaring **strategy** is not cosmetic. It is what causes the risk engine to apply spread margin instead of margining each leg independently. The identical legs submitted as **CUSTOM** get the short call margined as if naked, which typically demands an order of magnitude more buying power — often the difference between an order being accepted and rejected.

FOUR-LEG IRON CONDOR

```
{ "assetClass": "OPTION", "strategy": "IRON_CONDOR",
  "legs": [
    { "instrumentId": "inst_op_SPY_20261120P00520000", "side": "BUY_TO_OPEN", "ratio": 1 },
    { "instrumentId": "inst_op_SPY_20261120P00540000", "side": "SELL_TO_OPEN", "ratio": 1 },
    { "instrumentId": "inst_op_SPY_20261120C00600000", "side": "SELL_TO_OPEN", "ratio": 1 },
    { "instrumentId": "inst_op_SPY_20261120C00620000", "side": "BUY_TO_OPEN", "ratio": 1 }
  ],
  "quantity": "2", "orderType": "NET_CREDIT",
  "netPrice": "1.85", "priceEffect": "CREDIT", "timeInForce": "DAY" }
```

RESOLVING CONTRACTS

GET `/v1/instruments/{underlyingSymbol}/option-chain`

Requires `instruments:read` · always narrow with `expiration`

Option contracts carry a 21-character OSI symbol — `AAPL 261218C00250000` : six characters of root (space-padded), six digits of `YYMMDD`, `C` or `P`, then eight digits of strike × 1000. It is a useful checksum on your own logic, but the `instrumentId` is what you place orders against.

5.5 Previewing an order

POST `/v1/accounts/{accountId}/orders/preview`

Requires `orders:write` · creates nothing, takes no idempotency key

Runs the full validation and pricing pipeline without routing. Same request body as `placeOrder`. Use it to power a confirmation screen.

```

→ 200 OK
{ "valid": false,
  "estimatedCost": "123750.00",
  "commission": "0.00",
  "fees": [ { "type": "REGULATORY", "amount": "0.00" } ],
  "totalEstimated": "123750.00",
  "buyingPowerBefore": "84360.20",
  "buyingPowerAfter": "-39389.80",
  "marginRequirement": "61875.00",
  "maxLoss": "3150.00", // defined-risk strategies; null if unbounded
  "warnings": [
    { "code": "EXCEEDS_BUYING_POWER", "severity": "ERROR",
      "message": "This order exceeds available stock buying power by 39,389.80 USD." },
    { "code": "CONCENTRATION", "severity": "WARNING",
      "message": "After this order, MSFT would be 67% of account equity." }
  ] }

```

`valid` is false when any warning has `severity: ERROR`. Submitting anyway will be rejected — show the warnings instead.

5.6 Amending and cancelling

PATCH `/v1/accounts/{accountId}/orders/{orderId}`

DELETE `/v1/accounts/{accountId}/orders/{orderId}`

Requires `orders:write` · If-Match required on both

Amendment uses JSON Merge Patch (RFC 7386). Only `quantity`, `limitPrice`, `stopPrice`, `trailingAmount`, `trailingPercent`, `netPrice`, `timeInForce`, and `goodTilDate` are amendable. Changing the instrument, the side, or the leg structure is a different order: cancel and place a new one.

```

PATCH /v1/accounts/acct_.../orders/ord_...
If-Match: W/"ord-7-1755441062"
Content-Type: application/merge-patch+json

{ "quantity": "200", "limitPrice": "413.25" }

```

New `quantity` must be at least what has already filled. Amendment is best-effort against a live venue: a `200` means the amendment was accepted for routing, and the order may still fill at the old price if that fill was already in flight.

Cancellation is idempotent — cancelling an already-cancelled order returns `200` with the cancelled order rather than an error.

5.7 Streaming order events and webhooks

GET /v1/accounts/{accountId}/orders/events

Requires `orders:read` · `text/event-stream`

A Server-Sent Events stream of order transitions and executions. Prefer this over polling.

```
id: evt_01HQ9E1A2B3C4D5E
event: execution
data: {"eventId":"evt_01HQ9E1A2B3C4D5E","eventType":"execution",
data: "orderId":"ord_01HQ9D4N7Q1S5U8X","status":"PARTIALLY_FILLED",
data: "execution":{"quantity":"100","price":"412.48","venue":"XNAS"}}
```

BEHAVIOR	DETAIL
Resume	Send <code>Last-Event-ID</code> on reconnect to replay from just after that event. Retention is 5 minutes.
Heartbeat	A comment frame every 15 seconds. Treat 45 seconds of silence as a dead connection and reconnect.
Delivery	At-least-once. Deduplicate on <code>eventId</code> .
Limit	5 concurrent streams per grant.

WEBHOOKS

For server-side integrations that would rather be called than hold a connection open, register a webhook and receive the same `OrderEvent` payload by `POST` .

- Signed with `Webhook-Signature` : HMAC-SHA256 over the **raw** request body, keyed by your webhook secret. Verify before parsing, and compare in constant time.
- Return `2xx` within 5 seconds. Non-2xx and timeouts are retried with exponential backoff for 24 hours.
- Delivery is at-least-once, so handlers must be idempotent on `eventId` .

VERIFY THE SIGNATURE OVER RAW BYTES

Compute the HMAC over the body exactly as received, before any JSON parse and re-serialize. Round-tripping through a parser reorders keys and normalizes whitespace, which changes the bytes and breaks the signature — and the usual "fix" for that is to skip verification, which is how forged fill events get accepted.

6 Code Examples

The brokerage publishes no SDK, so these examples build directly on each language's standard HTTP client. That is deliberate: it keeps the wire protocol visible, and it means nothing here goes stale when a vendor library does.

6.1 How the examples are organized

Complete, runnable sources live alongside this document. The listings below are excerpts; the files are the reference.

FILE	CONTENTS
examples/python/ — standard library only, Python 3.9+	
<code>pkce_auth.py</code>	PKCE flow, token store, refresh with rotation, revocation.
<code>brokerage_client.py</code>	HTTP client: auth, retry, idempotency, link following, pagination.
<code>accounts_examples.py</code>	All five accounts endpoints.
<code>trading_examples.py</code>	Stock and option orders, preview, amend, cancel.
examples/node/ — built-ins only, Node 18+	
<code>pkce-auth.mjs</code>	As above, using <code>node:crypto</code> and <code>node:http</code> .
<code>brokerage-client.mjs</code>	As above, plus an SSE consumer and exact decimal arithmetic over <code>BigInt</code> .
<code>accounts-examples.mjs</code> , <code>trading-examples.mjs</code>	As above.
examples/java/ — Java 17+, Jackson the only dependency	
<code>PkceAuth.java</code>	As above, using <code>java.net.http.HttpClient</code> .
<code>BrokerageClient.java</code>	As above, with <code>BigDecimal</code> throughout.
<code>AccountsExamples.java</code> , <code>TradingExamples.java</code>	As above.
<code>pom.xml</code>	Maven build. <code>mvn compile exec:java -Dexec.mainClass=...</code>

All three clients default to the **sandbox** base URL, and the trading examples default to `dryRun = true` — they preview without routing. Both are deliberate: reaching production, and actually placing an order, should each require an explicit edit.

6.2 Authorization: PKCE in three languages

The same four steps everywhere: generate the pair, build the authorization URL, verify `state`, exchange the code.

PYTHON

```
import base64, hashlib, secrets

def generate_pkce_pair():
    """32 random bytes → a 43-character verifier, the RFC 7636 minimum."""
    verifier = base64.urlsafe_b64encode(secrets.token_bytes(32)).decode().rstrip("=")
    digest = hashlib.sha256(verifier.encode("ascii")).digest()
    challenge = base64.urlsafe_b64encode(digest).decode().rstrip("=")
    return verifier, challenge

# `secrets`, never `random` – the verifier is the only thing stopping
# someone who intercepts the authorization code from redeeming it.

tokens = exchange_code(code, verifier) # POST /oauth2/token, no client_secret
```

NODE.JS

```
import { createHash, randomBytes, timingSafeEqual } from 'node:crypto';

export function generatePkcePair() {
  const codeVerifier = randomBytes(32).toString('base64url');
  const codeChallenge = createHash('sha256').update(codeVerifier).digest('base64url');
  return { codeVerifier, codeChallenge };
}

// Constant-time compare on `state` – habits matter more than this one leak.
function safeEqual(a, b) {
  const x = Buffer.from(a), y = Buffer.from(b);
  return x.length === y.length && timingSafeEqual(x, y);
}
```

JAVA

```

public static PkcePair generatePkcePair() {
    byte[] entropy = new byte[32];
    RANDOM.nextBytes(entropy);           // SecureRandom
    String verifier = b64url(entropy);

    byte[] digest = MessageDigest.getInstance("SHA-256")
        .digest(verifier.getBytes(StandardCharsets.US_ASCII));
    return new PkcePair(verifier, b64url(digest));
}

private static String b64url(byte[] raw) {
    return Base64.getUrlEncoder().withoutPadding().encodeToString(raw);
}

```

REFRESH WITH ROTATION — ALL THREE CLIENTS SHARE THIS SHAPE

Persist the new refresh token **before** using the new access token, atomically (write-temp-then-rename). On `invalid_grant`, stop retrying and re-authorize: the token is revoked, expired, or already spent, and there is no other recovery. Serialize concurrent refreshes behind a lock, or two threads will trip reuse detection and revoke the family.

6.3 Accounts: reading balances and positions

Note that all three follow the `balances` link rather than building the URL, and all three parse money into an exact decimal type.

PYTHON

```

from decimal import Decimal

accounts = client.get("/accounts")["items"]
account = accounts[0]

balances = client.follow(account, "balances") # HATEOAS, not string concat

print(f"Net liquidation: {Decimal(balances['netLiquidation']):,.2f}")
print(f"Stock BP:      {Decimal(balances['buyingPower']['stock']):,.2f}")
print(f"Option BP:     {Decimal(balances['buyingPower']['option']):,.2f}")

# A margin call is the one balance field worth alerting on.
if (call := (balances.get("margin") or {}).get("marginCall")):
    alert(f"{call['type']}: {call['amount']} due {call['dueDate']}")

# paginate() follows `next` links; it never constructs a cursor.
total = sum(Decimal(p["unrealizedPnl"])
            for p in client.paginate(client.link(account, "positions")))

```

NODE.JS

```

const { items: accounts } = await client.get('/accounts');
const account = accounts[0];
const balances = await client.follow(account, 'balances');

// Dec keeps values as strings and does exact arithmetic over BigInt.
// Number('412.50') would be fine; Number arithmetic would not be.
let totalUnrealized = '0';
for await (const p of client.paginate(client.link(account, 'positions'))) {
    totalUnrealized = Dec.add(totalUnrealized, p.unrealizedPnl);
}
console.log(`Unrealized: ${Dec.format(totalUnrealized)}`);

```

JAVA

```

JsonNode account = client.get("/accounts").path("items").get(0);
JsonNode balances = client.follow(account, "balances");

System.out.printf("Net liquidation: %,14.2f%n", decimal(balances, "netLiquidation"));
System.out.printf("Stock BP:          %,14.2f%n", decimal(balances, "buyingPower", "stock"));

BigDecimal total = client.paginate(client.linkOrThrow(account, "positions"), Options.none())
    .map(p -> decimal(p, "unrealizedPnl"))
    .reduce(BigDecimal.ZERO, BigDecimal::add);

```

6.4 Trading: placing a stock order

The pattern is identical across languages: generate the key once, preview, submit with the key, then confirm terminal state separately.

PYTHON

```

order = {
    "assetClass": "EQUITY",
    "side": "BUY",
    "instrumentId": resolve_equity(client, "MSFT")["instrumentId"],
    "quantity": "300", # strings, never floats
    "orderType": "LIMIT",
    "limitPrice": "412.50",
    "timeInForce": "DAY",
}

result = client.post(f"{account_url}/orders/preview", order)
if not result["valid"]:
    for w in result["warnings"]:
        print(f"[{w['severity']}] {w['message']}")
    return

# Generated ONCE, outside the client's retry loop. Regenerating it in the
# loop is the exact bug idempotency exists to prevent.
key = client.new_idempotency_key()
order = client.post(f"{account_url}/orders", order, idempotency_key=key)

print(order["orderId"], order["status"]) # NEW - routed, not filled

```

NODE.JS

```
const order = {
  assetClass: 'EQUITY', side: 'BUY',
  instrumentId: (await resolveEquity(client, 'MSFT')).instrumentId,
  quantity: '300', orderType: 'LIMIT', limitPrice: '412.50', timeInForce: 'DAY',
};

const preview = await client.post(`${accountUrl}/orders/preview`, order);
if (!preview.valid) return preview.warnings.forEach(w => console.log(w.message));

const idempotencyKey = client.newIdempotencyKey();
const placed = await client.post(`${accountUrl}/orders`, order, { idempotencyKey });
```

JAVA

```
ObjectNode order = MAPPER.createObjectNode();
order.put("assetClass", "EQUITY");
order.put("side", "BUY");
order.put("instrumentId", resolveEquity(client, "MSFT").path("instrumentId").asText());
order.put("quantity", "300");
order.put("orderType", "LIMIT");
order.put("limitPrice", "412.50");
order.put("timeInForce", "DAY");

JsonNode preview = client.post(accountUrl + "/orders/preview", order, Options.none());
if (!preview.path("valid").asBoolean()) { /* surface warnings */ return; }

String key = BrokerageClient.newIdempotencyKey();
JsonNode placed = client.post(accountUrl + "/orders", order,
    Options.none().idempotencyKey(key));
```

6.5 Trading: placing a multi-leg option order

A ten-lot vertical debit spread — twenty contracts — priced as one unit.

PYTHON

```
long_leg = resolve_option(client, "AAPL", "2026-12-18", "250.00", "CALL")
short_leg = resolve_option(client, "AAPL", "2026-12-18", "260.00", "CALL")

order = {
    "assetClass": "OPTION",
    "strategy": "VERTICAL", # this is what gets spread margin applied
    "legs": [
        {"instrumentId": long_leg["instrumentId"], "side": "BUY_TO_OPEN", "ratio": 1},
        {"instrumentId": short_leg["instrumentId"], "side": "SELL_TO_OPEN", "ratio": 1},
    ],
    "quantity": "10", # 10 SPREADS = 20 contracts
    "orderType": "NET_DEBIT",
    "netPrice": "3.15", # always positive
    "priceEffect": "DEBIT", # direction lives here, not in the sign
    "timeInForce": "DAY",
}
```

NODE.JS

```
const [longLeg, shortLeg] = await Promise.all([
  resolveOption(client, 'AAPL', '2026-12-18', '250.00', 'CALL'),
  resolveOption(client, 'AAPL', '2026-12-18', '260.00', 'CALL'),
]);

const order = {
  assetClass: 'OPTION', strategy: 'VERTICAL',
  legs: [
    { instrumentId: longLeg.instrumentId, side: 'BUY_TO_OPEN', ratio: 1 },
    { instrumentId: shortLeg.instrumentId, side: 'SELL_TO_OPEN', ratio: 1 },
  ],
  quantity: '10', orderType: 'NET_DEBIT',
  netPrice: '3.15', priceEffect: 'DEBIT', timeInForce: 'DAY',
};
```

JAVA

```
ObjectNode order = MAPPER.createObjectNode();
order.put("assetClass", "OPTION");
order.put("strategy", "VERTICAL");

ArrayNode legs = order.putArray("legs");
legs.add(leg(longLeg.path("instrumentId").asText(), "BUY_TO_OPEN", 1));
legs.add(leg(shortLeg.path("instrumentId").asText(), "SELL_TO_OPEN", 1));

order.put("quantity", "10");
order.put("orderType", "NET_DEBIT");
order.put("netPrice", "3.15");
order.put("priceEffect", "DEBIT");
order.put("timeInForce", "DAY");
```

6.6 Amend and cancel with optimistic concurrency

Read, check the affordance, mutate with `If-Match`, and handle 412 by re-reading rather than blind-retrying.

PYTHON

```
def amend_order(client, order, new_price):
    self_url = client.link(order, "self")
    fresh, headers = client.get_with_headers(self_url)

    # Branch on the link, not on the status string.
    if not client.can(fresh, "replace"):
        return fresh

    try:
        return client.patch(self_url, {"limitPrice": new_price},
                             if_match=headers.get("ETag"))
    except ApiError as exc:
        if exc.status != 412:
            raise
        # The order changed while we were deciding. Re-read and decide
        # again – never retry with the newer ETag unexamined.
        return client.get(self_url)
```

NODE.JS

```
const { body: fresh, headers } = await client.getWithHeaders(selfUrl);
if (!client.can(fresh, 'cancel')) return fresh; // already terminal

const cancelled = await client.delete(selfUrl, { ifMatch: headers.get('ETag') });
// PENDING_CANCEL, not CANCELLED – the venue has not acknowledged yet.
console.log(cancelled.status);
```

JAVA

```
Response current = client.getWithHeaders(selfUrl, Options.none());
JsonNode fresh = current.body();

if (!can(fresh, "replace")) return fresh;

try {
    ObjectNode patch = MAPPER.createObjectNode();
    patch.put("limitPrice", newPrice);
    return client.patch(selfUrl, patch, Options.none().ifMatch(current.headers().etag()));
} catch (ApiException e) {
    if (e.status() != 412) throw e;
    return client.get(selfUrl); //re-read, then decide again
}
```

STREAMING, IN THE NODE CLIENT

`brokerage-client.mjs` includes a complete SSE consumer: `streamOrderEvents()` is an async generator that reconnects automatically, resumes from `Last-Event-ID`, skips heartbeat comment frames, and reassembles multi-line `data:` fields. Deduplicate on `eventId` in your handler — delivery is at-least-once.

7 OpenAPI 3.2.0 Specification

Every endpoint in this document is described by a machine-readable OpenAPI 3.2.0 document. It is the normative artifact: where this prose and the specification disagree, the specification wins.

7.1 Where the specification lives

FILE	PURPOSE
openapi/brokerage-api.yaml	The source of truth. Hand-maintained, commented.
openapi/brokerage-api.json	Generated JSON rendering, for tooling that will not read YAML.
dev-portal/openapi/	The copy the developer portal serves and renders.

17 paths · 21 operations · 48 schemas · 1 webhook · validates clean against the official OpenAPI 3.2 meta-schema.

7.2 What 3.2.0 gives us that 3.1 did not

Five 3.2.0 additions earn their place in this document. Each is used because it lets the description state something that previously needed a vendor extension or a prose footnote.

`$SELF` — DOCUMENT IDENTITY

```
openapi: 3.2.0
$self: https://api.brokerage.example.com/openapi/brokerage-api.yaml
```

Gives the document a stable, self-assigned URI that also acts as the base for reference resolution — so `$ref` targets resolve identically whether the file is read from disk, a CDN, or a build pipeline. Previously the base URI was wherever the file happened to be fetched from, which made references environment-dependent.

HIERARCHICAL TAGS: SUMMARY, PARENT, KIND

```
tags:
  - name: accounts
    summary: Accounts
    kind: nav
  - name: balances
    summary: Balances
    parent: accounts      # nests under Accounts in navigation
    kind: nav
  - name: trade-enabled
    summary: Trade-enabled
    kind: audience       # an audience facet, not a nav node
```

The API description now expresses its own navigation hierarchy instead of delegating that to renderer-specific extensions like `x-tagGroups`. `kind` separates structural navigation from cross-cutting facets — here, which operations need a write grant.

THE QUERY METHOD

```
/instruments:
  get: # simple lookups by query string
  query: # structured screens with a request body – safe, idempotent, cacheable
```

HTTP QUERY is a safe, idempotent method that carries a request body. A structured instrument screen — exchanges, sectors, market-cap bounds, sort order — is too large and too nested for a query string, but it is still unambiguously a *read*. Before 3.2.0 the only way to describe it was as a `POST`, which misrepresents the semantics and defeats caching. A `POST /instrument-screener` alias remains for clients that cannot issue QUERY, notably browser `fetch()`.

ITEMSCHEMA FOR STREAMING

```
content:
  text/event-stream:
    itemSchema: # describes ONE event, not the whole stream
      type: object
      properties:
        id: { type: string }
        event: { type: string, enum: [order.accepted, execution, ...] }
        data:
          type: string
          contentType: application/json
          contentSchema: { $ref: '#/components/schemas/OrderEvent' }
```

Unlike `schema`, which would describe the entire stream as an array, `itemSchema` applies to each item independently — which is what a streaming consumer actually processes as bytes arrive. It makes the order event stream properly describable for the first time.

OAuth2MetadataUrl AND THE DEVICE FLOW

```
securitySchemes:
  oauth2:
    type: oauth2
    oauth2MetadataUrl: https://auth.brokerage.example.com/.well-known/oauth-authorization-server
    flows:
      authorizationCode: { ... }
      deviceAuthorization: # declarable for the first time in 3.2.0
        deviceAuthorizationUrl: ...
```

`oauth2MetadataUrl` points tooling at the RFC 8414 authorization server metadata document, so endpoints, supported PKCE methods, and grant types are *discovered* rather than hard-coded into the description and left to drift.

7.3 Document structure

```
brokerage-api.yml
├─ openapi: 3.2.0
├─ $self          document identity and reference base
├─ info           title, summary, description, license (SPDX identifier)
├─ servers        production and sandbox
├─ tags           hierarchical navigation (parent / kind)
├─ security       default: oauth2 with accounts:read
├─ paths
│  ├─ /           API root – the one bookmarkable URL
│  ├─ /accounts   account list
│  ├─ /accounts/{id} single account
│  ├─ /accounts/{id}/balances balances
│  ├─ /accounts/{id}/positions[//{positionId}] positions
│  ├─ /accounts/{id}/transactions[//{txnId}] transaction history
│  ├─ /accounts/{id}/orders GET status · POST place
│  ├─ /accounts/{id}/orders/preview validate without routing
│  ├─ /accounts/{id}/orders/events SSE stream (itemSchema)
│  ├─ /accounts/{id}/orders/{orderId} GET · PATCH · DELETE
│  ├─ /accounts/{id}/orders/{orderId}/executions fills
│  ├─ /instruments GET search · QUERY screen
│  ├─ /instrument-screener POST alias for QUERY
│  ├─ /instruments/{id} instrument detail
│  └─ /instruments/{symbol}/option-chain option contracts
├─ webhooks
│  └─ orderUpdated push alternative to SSE
└─ components
   ├─ securitySchemes oauth2 (authorizationCode + deviceAuthorization), dpop
   ├─ parameters      AccountId, OrderId, Cursor, Limit, IdempotencyKey, IfMatch
   ├─ headers          ETag, RateLimit, RateLimit-Policy, Retry-After, ...
   ├─ responses        Problem, BadRequest, Unauthorized, Forbidden, NotFound,
                       PreconditionFailed, PreconditionRequired, TooManyRequests
   └─ schemas          48 total – Order, OrderRequest (discriminated union),
                       Balances, Position, Transaction, Instrument, Problem, ...
```

A 3.1/3.2 DETAIL THAT SILENTLY BREAKS CLIENTS

OpenAPI 3.1 replaced its bespoke schema dialect with JSON Schema 2020-12, and `nullable: true` is **not** a JSON Schema keyword — it was a 3.0-ism. Left in a 3.1 or 3.2 document it is silently ignored, so a field you believe is nullable validates as non-nullable and your generated client throws on a legitimate `null`.

The correct spelling is a type union: `type: [string, "null"]`, or `oneOf: [{ $ref: ... }, { type: "null" }]` when the base type comes from a `$ref`. This specification uses the correct form throughout; it is worth grepping any 3.1+ document you inherit for `nullable`.

7.4 Validating and generating from the specification

VALIDATE

```
# Against the official 3.2 meta-schema – the authoritative check.
curl -sL -o oas32.json https://spec.openapis.org/oas/3.2/schema/2025-09-17
python3 - <<'PY'
import json, yaml
from jsonschema import Draft202012Validator
meta = json.load(open('oas32.json'))
doc = yaml.safe_load(open('openapi/brokerage-api.yaml'))
errors = list(Draft202012Validator(meta).iter_errors(doc))
print('VALID' if not errors else f'{len(errors)} error(s)')
for e in errors[:10]:
    print(' /' + '/'.join(map(str, e.absolute_path)), '::', e.message[:200])
PY

# Linting and style. Note: 3.2.0 support varies by tool and version –
# check before trusting a "valid" from anything but the meta-schema.
npx @redocly/cli lint openapi/brokerage-api.yaml
```

VALIDATE THE EXAMPLES TOO

A specification can be structurally valid and still carry examples that contradict their own schemas — which is worse than no examples, because integrators copy them. Walk every `examples` block, resolve its sibling `schema`, and validate the two against each other in CI. Doing exactly that on this document is what surfaced the `nullable` problem described above.

GENERATE CLIENTS

```
# Most generators do not yet target 3.2.0. Downgrade for generation only –  
# the 3.2-specific fields ($self, query, itemSchema, tag parent/kind) are  
# additive, so dropping them yields a valid 3.1 document.
```

```
npx @redocly/cli bundle openapi/brokerage-api.yaml -o build-3.1.yaml
```

```
npx @openapitools/openapi-generator-cli generate \
```

```
  -i build-3.1.yaml -g java    -o clients/java
```

```
npx @openapitools/openapi-generator-cli generate \
```

```
  -i build-3.1.yaml -g python -o clients/python
```

```
npx @openapitools/openapi-generator-cli generate \
```

```
  -i build-3.1.yaml -g typescript-fetch -o clients/typescript
```

AUDIT GENERATED CODE FOR DECIMAL HANDLING

Generators map `type: string` to a native string, which is correct here — but some apply a "helpful" numeric coercion when they see a numeric-looking `pattern` or `format`. Check that every price and quantity lands in

`BigDecimal`, `Decimal`, or a string, and never in `double`. See §3.3.

MOCK AND TEST

```
# Prism serves a mock from the spec, including the response examples.
```

```
npx @stoplight/prism-cli mock openapi/brokerage-api.yaml --port 4010
```

```
# Contract-test a running service against the description.
```

```
npx @stoplight/prism-cli proxy openapi/brokerage-api.yaml \
```

```
  https://api.sandbox.brokerage.example.com/v1 --errors
```

A Appendix: Error catalogue

Every problem `type` this API returns. The URI is the stable contract — branch on it, and treat `title` and `detail` as display text only.

All types are relative to `https://developer.brokerage.example.com/problems/`.

TYPE	STATUS	MEANING	EXTENSION FIELDS · WHAT TO DO
<code>validation-failed</code>	400	Schema validation failed.	<code>errors[]</code> with JSON Pointers. A bug in your code — fix and resubmit.
<code>invalid-token</code>	401	Token missing, expired, malformed, or revoked.	Refresh once and retry. If the refresh also fails, re-authorize.
<code>insufficient-scope</code>	403	The grant lacks a required scope.	<code>requiredScope</code> , <code>grantedScopes[]</code> . Re-authorize with the wider scope; do not retry.
<code>options-level-insufficient</code>	403	Strategy above the account's approval level.	<code>requiredLevel</code> , <code>currentLevel</code> . Tell the customer which level to apply for.
<code>account-not-found</code>	404	No such account, or it is outside this grant.	Deliberately ambiguous, so the API cannot be used to enumerate accounts.
<code>order-not-found</code>	404	No such order on this account.	Check the <code>accountId</code> in the path matches the order.
<code>account-restricted</code>	409	Compliance hold or unmet margin call.	Reads still work; trading does not. Not retryable.
<code>order-not-amendable</code>	409	Order is terminal, or an amendment is already in flight.	<code>currentStatus</code> . Re-read the order and re-decide.
<code>precondition-failed</code>	412	<code>If-Match</code> did not match the current <code>ETag</code> .	<code>currentETag</code> . Re-read and decide again — never blind-retry with the new validator.
<code>insufficient-buying-power</code>	422	Not enough buying power for this order.	<code>required</code> , <code>available</code> (each an amount + currency). Check you read the right buying-power field (§4.2).
<code>market-closed</code>	422	The requested session is not open.	<code>nextOpen</code> . Resubmit with <code>extended-Hours</code> or <code>GTC</code> , or queue until <code>nextOpen</code> .
<code>instrument-not-tradable</code>	422	Halted, delisted, or not supported.	<code>reason</code> . Not retryable without a different instrument.
<code>price-outside-collar</code>	422	Limit price too far from the current market.	<code>collarLow</code> , <code>collarHigh</code> . A guard against fat-finger entry.

TYPE	STATUS	MEANING	EXTENSION FIELDS · WHAT TO DO
idempotency-key-reuse	422	Key reused with a different body.	originalOrderId . Nothing was created. A bug: one key per intent.
day-trade-limit	422	Order would breach the pattern-day-trader rule.	dayTradesUsed , dayTradesRemaining .
rate-limit-exceeded	429	Quota exhausted.	retryAfterSeconds . Back off with full jitter (§3.7).
venue-unavailable	503	Upstream venue or clearing system down.	Retryable with backoff. Order state is unknown — reconcile before resubmitting.

B Appendix: Enumerations

THESE LISTS GROW

New enum members are shipped as backward-compatible changes, without a version bump. Your deserializer must not throw on an unrecognized value — treat an unknown `OrderStatus` as working and rely on `_links` to know what actions are available (§3.8).

B.1 Time in force

VALUE	BEHAVIOR
DAY	Expires at the close of the regular session.
GTC	Good till cancelled. Persists across sessions, typically capped at 90 days.
GTD	Good till a date. Requires <code>goodTilDate</code> .
IOC	Immediate or cancel. Fills whatever it can at once; cancels the remainder.
FOK	Fill or kill. Fills entirely and immediately, or cancels entirely. No partials.
OPG	At the open. Participates in the opening auction only.
CLS	At the close. Participates in the closing auction only.

B.2 Order types

VALUE	REQUIRES	NOTES
MARKET	—	Execution guaranteed, price is not.
LIMIT	<code>limitPrice</code>	Price guaranteed, execution is not.
STOP	<code>stopPrice</code>	Becomes a market order when the stop trips.
STOP_LIMIT	<code>stopPrice</code> , <code>limitPrice</code>	Becomes a limit order when the stop trips. Can go unfilled in a fast market.
TRAILING_STOP	<code>trailingAmount</code> or <code>trailingPercent</code>	Stop follows the high-water mark. The two trail fields are mutually exclusive.
TRAILING_STOP_LIMIT	trail + <code>limitPrice</code>	As above, triggering a limit order.
NET_DEBIT	<code>netPrice</code> , <code>priceEffect</code>	Multi-leg option combination, priced as a unit. Cash paid.

VALUE	REQUIRES	NOTES
NET_CREDIT	netPrice , priceEffect	Multi-leg combination. Cash received.
NET_EVEN	priceEffect	Multi-leg combination at zero net cost.

B.3 Sides

ASSET CLASS	VALUES
Equity	BUY , SELL , SELL_SHORT , BUY_TO_COVER
Option	BUY_TO_OPEN , BUY_TO_CLOSE , SELL_TO_OPEN , SELL_TO_CLOSE

B.4 Option strategies

VALUE	LEGS	MINIMUM APPROVAL LEVEL
SINGLE	1	LEVEL_2 to open long; LEVEL_4 to open naked short
COVERED_CALL	1	LEVEL_1 — requires 100 shares held per contract
VERTICAL , CALENDAR , DIAGONAL	2	LEVEL_3
STRADDLE , STRANGLE , COLLAR	2–3	LEVEL_3
BUTTERFLY	3–4	LEVEL_3
IRON_CONDOR	4	LEVEL_3
CUSTOM	1–4	Margined leg by leg — usually requires far more buying power

B.5 Transaction types

TRADE	DIVIDEND	INTEREST	FEE	DEPOSIT	WITHDRAWAL	TRANSFER	ASSIGNMENT	EXERCISE	EXPIRATION
CORPORATE_ACTION	TAX	JOURNAL							

C Appendix: Glossary

Buying power

What the account can spend. Differs by product — stock, option, and day-trading buying power are three separate numbers in a margin account (§4.2).

Contract multiplier

Shares of the underlying per option contract. Normally 100; adjusted contracts, following a split or merger, differ. Multiply option quantity by this for share-equivalent exposure.

DPoP

Demonstrating Proof of Possession (RFC 9449). Binds an access token to a key pair so a stolen token is inert without the private key (§2.9).

Good-faith violation

In a cash account, buying with unsettled proceeds and selling before they settle. Repeated violations restrict the account to settled cash.

HATEOAS

Hypermedia as the Engine of Application State. Clients discover available actions from links in the representation rather than from hard-coded URLs (§3.1).

Idempotency key

A client-generated UUID identifying one order *intent*, making retries safe. One key per intent, reused across every retry of that intent (§3.5).

Liquidity (added / removed)

Whether a fill posted to the book and waited (added) or crossed the spread (removed). Drives exchange rebates and fees.

Maintenance requirement

Minimum equity that must be held against margin positions. Falling below it triggers a margin call.

Net liquidation value

Total account value if every position were liquidated at current marks.

OSI symbol

The 21-character options contract identifier: `AAPL 261218C00250000` — six characters of root (space-padded), `YYMMDD`, `C / P`, then strike $\times 1000$ in eight digits.

Pattern day trader (PDT)

An account making four or more day trades in five business days. Subject to a 25,000 USD minimum equity requirement.

PKCE

Proof Key for Code Exchange (RFC 7636). Binds an authorization code to the client instance that requested it. Mandatory under OAuth 2.1 (§2.4).

Price effect

Whether a multi-leg option order's net price is paid (`DEBIT`) or received (`CREDIT`). Carries the direction, so `netPrice` is always positive.

Problem details

RFC 9457. The machine-readable error format used throughout this API (§3.2).

Refresh token rotation

Issuing a new refresh token on every use and invalidating the old one. Replaying a spent token is treated as theft and revokes the whole family (§2.7).

Settlement date

When cash and securities actually change hands. T+1 for US equities as of May 2024.

Strategy unit

One instance of a multi-leg option combination. A `quantity` of 10 on a two-leg vertical means ten spreads — twenty contracts (§5.4).

Time in force

How long an order remains working before it expires. See Appendix B.1.

Brokerage REST API — Developer Specification v1.0.0

Companion developer portal: `brokerage-apis/dev-portal/index.html`

Machine-readable specification: `brokerage-apis/openapi/brokerage-api.yaml`

Runnable examples: `brokerage-apis/examples/{java,node,python}/`